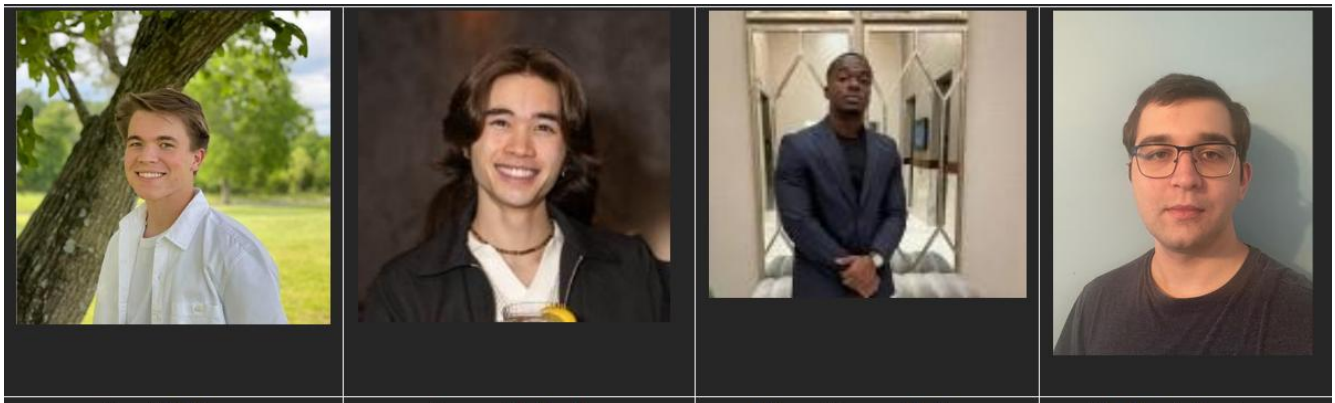
SP-14 Green Chess AI

A Software Design Document Template

4850-01
Spring 2026
Professor Perry
February 8, 2026

SP-14 Green Chess AI



Brett Talley	Kenneth Burke	Marc-Henry Moise	Vladimir Ryzhov
706-483-9553	925-822-4937	786-617-2013	470-775-1900

Table of Contents

1. INTRODUCTION AND OVERVIEW **3**

2. DESIGN CONSIDERATIONS **3**

- 2.1 ASSUMPTIONS AND DEPENDENCIES 3
- 2.2 GENERAL CONSTRAINTS 3
- 2.3 DEVELOPMENT METHODS 4

3. ARCHITECTURAL STRATEGIES **4**

4. SYSTEM ARCHITECTURE **4**

5. DETAILED SYSTEM DESIGN **6**

- 5.1 CLASSIFICATION 6
- 5.2 DEFINITION 6
- 5.3 CONSTRAINTS 6
- 5.4 RESOURCES 6
- 5.5 INTERFACE/EXPORTS 6

1. Introduction and Overview

Create a self-playing AI Chess Engine that offers both player vs player and player vs AI game modes. You may use any technology.

Objective:
Create a working chess game which follows the standard rules of chess. The chess has to have two game modes: Player vs Player (PvP) and Player vs Artificial Intelligence (PvAI). In case of abundance of time, we might decide to pursue the objective of adjustable difficulty for the AI game mode.

Code in Python using Pygame Module.

2. Design Considerations

– *2.1 Assumptions and Dependencies*

- Python with the Pygame module will be used to develop the software, the use of the software will be accessible through an executable.
- The software will be available for use on both Windows and UNIX operating systems.
- End user will only need knowledge on how to play chess. If they decide to play PvP chess, it is assumed that they are playing locally on the same system. If they decide to play against the AI, they will be able to make their move and the AI will respond with a move of it's own.
- Possible changes in functionality may include an option for elo adjustment against the AI.

– *2.2 General Constraints*

No limitations exist across hardware, software, and end user environments. Data repository will be kept locally within the system. As per performance, the AI must make their moves relatively quickly.

- Other requirements:
  General Requirements:
  - No illegal moves, no phasing through pieces, etc.
  - En passant, castling

  PvP Requirements:
  - 2 players
  - Can only move their respective pieces

  PvAI Requirements:

- Engine automatically generates the best move
- If time allows it, rank the moves
- Ranking the moves allows for adjustable difficulty

– *2.3 Development Methods*

The development of the software product will follow an iterative methodology. This means that we will begin by simply making the game of chess playable, and then iterate by adding full move legality, local PvP play, and an AI opponent. Software will be developed in Python with the Pygame module, but distributed in the form of an executable so end users need not know how to program to use the software.

The software will support Windows as well as UNIX-based operating systems. More rigid methodologies like Waterfall were evaluated but will not be used due to the gameplay and interaction requirements changing over the course of development. The AI opponent will follow a standard minimax algorithm with alpha–beta pruning, and will make a move in response to each of the player's moves. End users will only need to know how to play chess. PvP mode will assume two players are playing on the same computer locally.

3. Architectural Strategies

Architecturally, the system will include one shared chess core (board state, rules, movement validation, etc.) for use by both local PvP and AI chess play. Doing this should enforce shared behavior and prevent duplicated logic between the two approaches. Implementation will be in Python via Pygame. This allows for quick iteration and easy packaging into an executable, at the cost of advanced graphics and peak performance. Support platforms will be Windows and UNIX-style operating systems.

UI will be based on click-to-move actions. This was selected due to its straightforward nature and minimal learning curve. AI will be swappable modules that conform to a standard interface. Standard minimax with alpha–beta pruning will be implemented, allowing general difficulty adjustment and easier swapping of a better AI later. Invalid actions will simply be rejected with informative text (versus trying to recover from invalid actions). Gameplay will be strictly single player versus and local. No networking, threads, or database support will be added. This avoids complexity in other areas.

For expansion, the goal is to only add modules that extend behavior in supported ways. Examples include support for stronger AI, optional save/load, or UI polish.

4. System Architecture

At the heart of the system is a Chess Model object that knows about all of the game logic/state. This ensures that there is only one authoritative game state available to all of the other objects interacting with the game. GameModel encapsulates logic about how a game starts, ends, and switches turns. Board maintains the board array, its dimensions, and which side's turn it is.

Players, whether they be human or AI issue a Move (fromPiece, toPosition). The AIPlayer looks at the current Board state and decides upon a Move based on its internal decision making logic. The Move is then passed into the same Move execution workflow as if a human player had issued the move. Validation logic was broken out into an abstract Piece class. The subclasses (Pawn, Rook, Knight, Bishop, Queen, King) are then responsible for how they can move based on their type as well as specifying any special rules (ex. hasMoved).

5. Detailed System Design

– *5.1 Classification*

The system will be structured as a single desktop application compiled into a single executable file. Within the application there will be multiple cooperating classes that model the game logic, board state, player IO, and AI.

– *5.2 Definition*

This system will allow full chess gameplay, enabling local PvP as well as versus computer play. The system will control game flow, game board, movement validation, and AI calculations and will enforce all normal rules of chess.

– *5.3 Constraints*

The system will run locally on one machine and will not involve networking or external databases. Moves should be checked before they are performed and illegal moves will be rejected safely. The system should never hang while a game is in progress. AI time should be limited as necessary to ensure responsiveness.

– *5.4 Resources*

A description of any and all resources that are managed, affected, or needed by this entity. Resources are entities external to the design such as memory, processors, printers, databases, or a software library. This should include a discussion of any possible race conditions and/or deadlock situations, and how they might be resolved.

– *5.5 Interface*

The system will present a user-interface for player interactions. Players will be able to select pieces, move them, and receive updates from the game. Behind the scenes, the system will expose services that allow moves to start games, make moves, validate them against rules, and trigger AI play.